Pseudocode conventions

This set follows the pseudocode conventions of *Introduction to Algorithms* (Cormen, Leiserson, Rivest, Stein), 4th edition.

- **Procedures.** Every algorithm is a named procedure with parameters, written `NAME-IN-CAPITALS(a, b)`. There is no `START`, `END`, `READ` or `PRINT`. Data comes in through the parameters, and the answer goes out with `return`.
- **Indentation shows structure.** A block belongs to the line above it because it is indented under it. There are no `begin/end` or `END IF` lines.
- **Assignment and comparison.** `x = 5` assigns. `x == 5` tests equality. Also `≠` `≤` `≥`. (Older editions write assignment as `x ← 5` and equality as a single `=`.)
- **Logic.** `and`, `or`, `not`, with values `TRUE` and `FALSE`. `NIL` means "no value".
- **Loops and decisions.** `for i = 1 to n`, `for i = n downto 1`, `while condition`, `if ... elseif ... else`.
- **Arithmetic.** `n mod 2` is the remainder. `⌊n / 10⌋` is division rounded down.
- **Lists.** `A[i]` is one element, `A.length` is the number of elements, and indexing starts at 1. `let A[1:n] be a new list` creates an list. A string is treated as an list of characters.
- **Other statements.** `exchange A[i] with A[j]` swaps two elements. `PUSH(S, x)`, `POP(S)` and `STACK-EMPTY(S)` work on a stack S. `//` starts a comment.
- **Parameters are passed by value.** Changing a number parameter inside a procedure does not change the caller's variable. Changes to the elements of an list do show up in the caller.

Part A: Question paper

Write a procedure for each task below.

Very easy (Q1 to Q5)

1. `SUM-OF-TWO(a, b)`: return the sum of two numbers.
2. `EVEN-OR-ODD(n)`: return "Even" or "Odd" for a whole number.
3. `LARGER-OF-TWO(a, b)`: return the larger of two numbers.
4. `CELSIUS-TO-FAHRENHEIT(c)`: return a Celsius temperature in Fahrenheit.
5. `NUMBERS-UP-TO(n)`: return an list holding the numbers 1 to n.

Easy (Q6 to Q10)

6. `SUM-OF-FIRST-N(n)`: return 1 + 2 + ... + n.
7. `FACTORIAL(n)`: return n!.
8. `COUNT-VOWELS(w)`: return the number of vowels in a word.
9. `FIND-LARGEST(A)`: return the largest number in an list.
10. `COUNT-DIGITS(n)`: return how many digits a positive whole number has.

Medium (Q11 to Q18)

11. `IS-PRIME(n)`: return TRUE if n is prime, otherwise FALSE.
12. `FIBONACCI(n)`: return an list holding the first n Fibonacci numbers.

13. `IS-PALINDROME(w)`: return TRUE if a word reads the same backwards.
14. `SUM-OF-DIGITS(n)`: return the sum of the digits of a positive whole number.
15. `SECOND-LARGEST(A)`: return the second largest of distinct numbers in an list.
16. `LINEAR-SEARCH(A, v)`: return the position of v in A, or NIL if absent.
17. `BUBBLE-SORT(A)`: sort the list A in ascending order, in place.
18. `GCD(a, b)`: return the greatest common divisor of two positive numbers.

Hard (Q19 and Q20)

19. `BINARY-SEARCH(A, v)`: A is sorted. Return the position of v, or NIL if absent.
20. `BALANCED(s)`: s holds only the characters `( ) [ ] { }`. Using a stack, return TRUE if the brackets are balanced, otherwise FALSE.

Part B: Worked answers

Each answer has five parts: (1) input and output, (2) how to think about it, (3) the algorithm in plain English, (4) the pseudocode, and (5) a trace on a sample problem.

Very easy

Q1. Sum of two numbers

1. Input and output

- Input: two numbers, a and b (the parameters).
- Output: one number, a + b (the return value).

2. How to think about it Ask three things: What am I given? What do I want back? What connects them? Here the connection is addition. The given values arrive as named parameters, and the answer leaves through `return`.

3. Algorithm in plain English

1. Receive the two numbers a and b.
2. Add them.
3. Return the result.

4. Pseudocode

```
SUM-OF-TWO(a, b)
    sum = a + b
    return sum
```

5. Trace: SUM-OF-TWO(12, 8)

```
a = 12, b = 8
sum = 12 + 8 = 20
return 20
```

Q2. Even or odd

1. Input and output

- Input: one whole number n.
- Output: the text "Even" or "Odd".

2. How to think about it What makes a number even? It divides by 2 with nothing left over. So the real question is "what is the remainder when I divide by 2?" The operator `mod` answers that. Remainder 0 means even. Anything else means odd. Two possible outcomes need an `if ... else`.

3. Algorithm in plain English

1. Receive n.
2. Find the remainder of n divided by 2.
3. If the remainder is 0, return "Even".
4. Otherwise, return "Odd".

4. Pseudocode

```
EVEN-OR-ODD(n)
    if n mod 2 == 0
        return "Even"
    else
        return "Odd"
```

5. Trace: EVEN-OR-ODD(7)

```
n = 7
7 mod 2 = 1
Is 1 == 0 ?  No, so take the else branch
return "Odd"
```

Q3. Larger of two numbers

1. Input and output

- Input: two numbers, a and b.
- Output: the larger of the two, or the text "Equal" if they are the same.

2. How to think about it Do what you would do on paper: compare them. There are three possible outcomes (a bigger, b bigger, equal), so we need a decision with three branches. Always ask "what if they are equal?" A good programmer handles that case instead of ignoring it.

3. Algorithm in plain English

1. Receive a and b.
2. If a is greater than b, return a.
3. Otherwise, if b is greater than a, return b.
4. Otherwise they must be equal, so return "Equal".

4. Pseudocode

```
LARGER-OF-TWO(a, b)
    if a > b
```

```
        return a
    elseif b > a
        return b
    else
        return "Equal"
```

5. Trace: LARGER-OF-TWO(15, 9)

```
Is 15 > 9 ?  Yes, so return 15
(the remaining branches are never reached)
```

Q4. Celsius to Fahrenheit

1. Input and output

- Input: temperature c in Celsius.
- Output: the same temperature in Fahrenheit.

2. How to think about it This question needs a formula, not a decision. If you do not remember it, find it first: f = (c × 9 / 5) + 32. Pseudocode only turns a known formula into steps. Remember that multiplication and division happen before addition.

3. Algorithm in plain English

1. Receive c.
2. Multiply c by 9, divide by 5, then add 32.
3. Return the answer.

4. Pseudocode

```
CELSIUS-TO-FAHRENHEIT(c)
    f = c * 9 / 5 + 32
    return f
```

5. Trace: CELSIUS-TO-FAHRENHEIT(100)

```
c = 100
c * 9    = 900
900 / 5  = 180
180 + 32 = 212
return 212
```

Q5. Numbers 1 to n

1. Input and output

- Input: a number n.
- Output: an list A with n elements: 1, 2, 3, ..., n.

2. How to think about it You would not write n separate assignment lines, because n is not known in advance. When the same action repeats, use a loop. A loop needs three things: a counter that starts somewhere (1), a stopping point (n), and a step (add 1 each time). `for` bundles all three. Because the procedure cannot print, it builds an list and returns it.

3. Algorithm in plain English

1. Receive n and create an empty list with room for n numbers.
2. Go through positions 1 to n. In position i, store the value i.
3. Return the list.

4. Pseudocode

```
NUMBERS-UP-TO(n)
    let A[1:n] be a new list
    for i = 1 to n
        A[i] = i
    return A
```

5. Trace: NUMBERS-UP-TO(4)

i	Action	A so far
1	A[1] = 1	[1]
2	A[2] = 2	[1, 2]
3	A[3] = 3	[1, 2, 3]
4	A[4] = 4	[1, 2, 3, 4]

The loop ends after i = 4. Returns [1, 2, 3, 4].

Easy

Q6. Sum of the first n natural numbers

1. Input and output

- Input: a positive whole number n.
- Output: 1 + 2 + ... + n.

2. How to think about it Do it by hand for n = 4: 1, then 1+2 = 3, then 3+3 = 6, then 6+4 = 10. You keep a running total and add one new number each time. In code, the running total is a variable that starts at 0 (adding 0 changes nothing) and grows inside a loop. This pattern is called an **accumulator**.

3. Algorithm in plain English

1. Receive n and set total to 0.
2. For each number from 1 up to n, add it to total.
3. Return total.

4. Pseudocode

```
SUM-OF-FIRST-N(n)
    total = 0
    for i = 1 to n
        total = total + i
    return total
```

5. Trace: SUM-OF-FIRST-N(5)

i	total before	total after
1	0	1
2	1	3
3	3	6
4	6	10
5	10	15

Returns 15.

Q7. Factorial

1. Input and output

- Input: a whole number n (0 or more).
- Output: n! = 1 × 2 × 3 × ... × n. By definition 0! = 1.

2. How to think about it It is the same accumulator idea as Q6, but we multiply instead of add. That changes the starting value. Adding starts from 0 because adding 0 changes nothing. Multiplying must start from 1 because multiplying by 1 changes nothing. Starting a product at 0 would make every answer 0. A bonus: if n = 0 the loop never runs and the result stays 1, which is correct.

3. Algorithm in plain English

1. Receive n and set result to 1.
2. For each number from 1 up to n, multiply result by it.
3. Return result.

4. Pseudocode

```
FACTORIAL(n)
    result = 1
    for i = 1 to n
        result = result * i
    return result
```

5. Trace: FACTORIAL(5)

i	result before	result after
1	1	1
2	1	2
3	2	6
4	6	24

i	result before	result after
5	24	120

Returns 120.

Q8. Count the vowels in a word

1. Input and output

- Input: a word w (an list of characters).
- Output: the number of vowels (a, e, i, o, u) in it.

2. How to think about it A person reads the word letter by letter and makes a tally mark for every vowel. The computer does the same. Visit each letter (a loop), ask "is it a vowel?" (a decision), and if yes add 1 to a counter. This is a **counter** pattern: like an accumulator, but it only changes when a condition is true.

3. Algorithm in plain English

1. Receive the word and set count to 0.
2. Look at each letter, one at a time.
3. If the letter is a, e, i, o or u, add 1 to count.
4. After the last letter, return count.

4. Pseudocode

```
COUNT-VOWELS(w)
    count = 0
    for i = 1 to w.length
        ch = w[i]
        if ch == "a" or ch == "e" or ch == "i" or ch == "o" or ch == "u"
            count = count + 1
    return count
```

5. Trace: COUNT-VOWELS("education")

i	ch	Vowel?	count
1	e	yes	1
2	d	no	1
3	u	yes	2
4	c	no	2
5	a	yes	3
6	t	no	3
7	i	yes	4
8	o	yes	5

i	ch	Vowel?	count
9	n	no	5

Returns 5.

Q9. Largest number in an list

1. Input and output

- Input: an list A of numbers (A.length gives how many).
- Output: the largest number in A.

2. How to think about it Imagine the numbers are written on cards and you may look at only one at a time. You pick up the first card and say, "this is the biggest so far." For every later card, compare it with your champion. If the new card is bigger, it becomes the new champion. When the cards run out, the champion is the answer. The trick is to **start with the first item**, not with 0, because the list may contain only negative numbers.

3. Algorithm in plain English

1. Receive the list. Let largest be the first number.
2. Look at the remaining numbers one by one.
3. If a number is bigger than largest, replace largest with it.
4. Return largest.

4. Pseudocode

```
FIND-LARGEST(A)
    largest = A[1]
    for i = 2 to A.length
        if A[i] > largest
            largest = A[i]
    return largest
```

5. Trace: FIND-LARGEST([4, 9, 2, 11, 7])

i	A[i]	A[i] > largest?	largest
(start)			4
2	9	yes	9
3	2	no	9
4	11	yes	11
5	7	no	11

Returns 11.

Q10. Count the digits of a number

1. Input and output

- Input: a positive whole number n.
- Output: how many digits n has.

2. How to think about it We cannot just look at a number and "see" its digits, but we can chop digits off. Dividing a whole number by 10 and rounding down removes its last digit (4052 becomes 405). So count how many times you can chop before nothing is left. Each chop is one digit. "Until nothing is left" tells you to use a `while` loop, because we do not know in advance how many rounds there will be. Since parameters are passed by value, changing n inside the procedure does not harm the caller's copy.

3. Algorithm in plain English

1. Receive n and set count to 0.
2. As long as n is greater than 0, remove its last digit (divide by 10, rounding down) and add 1 to count.
3. Return count.

4. Pseudocode

```
COUNT-DIGITS(n)
    count = 0
    while n > 0
        n = ⌊n / 10⌋
        count = count + 1
    return count
```

5. Trace: COUNT-DIGITS(4052)

n > 0?	n after ⌊n / 10⌋	count
4052 > 0 yes	405	1
405 > 0 yes	40	2
40 > 0 yes	4	3
4 > 0 yes	0	4
0 > 0 no	stop	4

Returns 4.

Medium

Q11. Prime number check

1. Input and output

- Input: a whole number n.
- Output: TRUE if n is prime, otherwise FALSE.

2. How to think about it A prime has exactly two divisors: 1 and itself. So to prove a number is *not* prime, we only need to find one number between 2 and n-1 that divides it exactly. Think of it as a search for a "culprit". The moment one is found, the answer is settled, so we can `return FALSE` immediately. If the loop ends without finding a culprit, then there was none, and we return TRUE. Special case: numbers below 2 are never prime.

3. Algorithm in plain English

1. Receive n. If n is less than 2, return FALSE.
2. Try every number i from 2 to n-1.
3. If i divides n exactly, n is not prime, so return FALSE at once.
4. If no i divided n, return TRUE.

4. Pseudocode

```
IS-PRIME(n)
    if n < 2
        return FALSE
    for i = 2 to n - 1
        if n mod i == 0
            return FALSE
    return TRUE
```

5. Trace: IS-PRIME(9)

i	9 mod i	Divides?	Action
2	1	no	continue
3	0	yes	return FALSE

Returns FALSE, and the loop never reaches i = 4. For IS-PRIME(7), no i from 2 to 6 divides 7, the loop ends, and the procedure returns TRUE.

Q12. Fibonacci series

1. Input and output

- Input: n, the number of terms (n ≥ 2).
- Output: an list F of n numbers: 0, 1, 1, 2, 3, 5, 8, ...

2. How to think about it The rule is: every term is the sum of the two terms before it. The first two terms (0 and 1) cannot follow the rule because nothing comes before them, so we set them by hand. After that, each new term is built from the list itself: F[i] = F[i-1] + F[i-2]. Storing the terms in an list means we never need to shuffle temporary variables.

3. Algorithm in plain English

1. Receive n and create an list F with room for n numbers.
2. Set F[1] to 0 and F[2] to 1.
3. For each position from 3 to n, store the sum of the two previous terms.
4. Return F.

4. Pseudocode

```
FIBONACCI(n)
    let F[1:n] be a new list
    F[1] = 0
    F[2] = 1
    for i = 3 to n
        F[i] = F[i - 1] + F[i - 2]
    return F
```

5. Trace: FIBONACCI(6)

i	F[i-1]	F[i-2]	F[i]
(start)			F = [0, 1, _, _, _, _]
3	1	0	1
4	1	1	2
5	2	1	3
6	3	2	5

Returns [0, 1, 1, 2, 3, 5].

Q13. Palindrome check

1. Input and output

- Input: a word w (an list of characters).
- Output: TRUE if w reads the same forwards and backwards, otherwise FALSE.

2. How to think about it A palindrome reads the same in both directions ("radar"). You do not need to build a reversed copy. Put one finger on the first letter and one on the last. They must match. Then move both fingers inward and compare again. If any pair differs, return FALSE at once. If the fingers meet or cross with no mismatch, return TRUE. These two fingers are called **two pointers**.

3. Algorithm in plain English

1. Receive the word. Put left at position 1 and right at the last position.
2. While left is before right: if the two letters differ, return FALSE.
3. Otherwise move left one step right and right one step left.
4. If the loop finishes, return TRUE.

4. Pseudocode

```
IS-PALINDROME(w)
    left = 1
    right = w.length
    while left < right
        if w[left] ≠ w[right]
            return FALSE
        left = left + 1
```

```
        right = right - 1
    return TRUE
```

5. Trace: IS-PALINDROME("radar")

left	right	w[left]	w[right]	Result
1	5	r	r	equal, move on
2	4	a	a	equal, move on
3	3			left < right is false, loop ends

Returns TRUE. For "hello": h and o differ on the first comparison, so the procedure returns FALSE immediately.

Q14. Sum of digits

1. Input and output

- Input: a positive whole number n.
- Output: the sum of its digits (2394 gives 2+3+9+4 = 18).

2. How to think about it Combine two tricks you already know. `n mod 10` **gives** the last digit (2394 mod 10 = 4). `⌊n / 10⌋` **removes** the last digit (2394 becomes 239). So peel off digits one at a time, adding each to a running total, until nothing is left. This is Q10 (chopping digits) plus Q6 (accumulator).

3. Algorithm in plain English

1. Receive n and set total to 0.
2. While n is greater than 0: take the last digit and add it to total, then remove the last digit from n.
3. Return total.

4. Pseudocode

```
SUM-OF-DIGITS(n)
    total = 0
    while n > 0
        digit = n mod 10
        total = total + digit
        n = ⌊n / 10⌋
    return total
```

5. Trace: SUM-OF-DIGITS(2394)

n	digit = n mod 10	total	n after ⌊n / 10⌋
2394	4	4	239
239	9	13	23
23	3	16	2
2	2	18	0

n is now 0, so the loop stops. Returns 18.

Q15. Second largest number

1. Input and output

- Input: an list A of at least 2 distinct numbers.
- Output: the second largest number.

2. How to think about it Extend the "champion" idea from Q9. Now keep **two** records: the champion (first) and the runner-up (second). For each new number there are three cases: it beats the champion (the old champion becomes the runner-up and the number becomes champion), or it only beats the runner-up (it replaces the runner-up), or it beats neither (ignore it). Handle the first two numbers up front so both records start with real values.

3. Algorithm in plain English

1. Compare the first two numbers: the bigger is first, the smaller is second.
2. For each remaining number: if it is bigger than first, move first down to second and make this number first. Otherwise, if it is bigger than second, make it second.
3. Return second.

4. Pseudocode

```
SECOND-LARGEST(A)
    if A[1] > A[2]
        first = A[1]
        second = A[2]
    else
        first = A[2]
        second = A[1]
    for i = 3 to A.length
        if A[i] > first
            second = first
            first = A[i]
        elseif A[i] > second
            second = A[i]
    return second
```

5. Trace: SECOND-LARGEST([7, 15, 9, 20, 12])

Step	A[i]	Decision	first	second
start	7, 15	7 > 15 is false, so else branch	15	7
i = 3	9	not > 15, but > 7	15	9
i = 4	20	> 15	20	15
i = 5	12	not > 20, not > 15	20	15

Returns 15.

Q16. Linear search

1. Input and output

- Input: an list A and a value v to look for.
- Output: the position (1 to A.length) where v is first found, or NIL if it is absent.

2. How to think about it Like looking for a name on an unsorted attendance sheet: start at the top and check each entry until you find it or reach the end. The moment you find it, return that position. Only after checking every entry can you say "not here", so `return NIL` goes after the loop, not inside it.

3. Algorithm in plain English

1. Receive the list and the value.
2. Go through positions 1 to the last.
3. At the first position whose element equals v, return that position.
4. If the loop ends without a match, return NIL.

4. Pseudocode

```
LINEAR-SEARCH(A, v)
    for i = 1 to A.length
        if A[i] == v
            return i
    return NIL
```

5. Trace: LINEAR-SEARCH([8, 3, 15, 6], 15)

i	A[i]	A[i] == 15?	Action
1	8	no	continue
2	3	no	continue
3	15	yes	return 3

Returns 3.

Q17. Bubble sort

1. Input and output

- Input: an list A of numbers.
- Output: nothing is returned. The same list A is rearranged into ascending order (in place), and the caller sees the change because list elements are shared.

2. How to think about it Look only at **neighbours**. If two neighbours are in the wrong order (left bigger than right), swap them. One pass from left to right carries the biggest number all the way to the end, like a bubble rising. After pass 1 the last position is correct. After pass 2 the last two are correct, and so on. So after n-1 passes everything is sorted, and each pass can stop one position earlier than the last. CLRS gives `exchange A[j] with A[j + 1]` as a single statement for the swap.

3. Algorithm in plain English

1. Repeat for pass = 1 up to A.length - 1.
2. In each pass, compare each pair of neighbours from position 1 up to A.length - pass.
3. If the left one is larger than the right one, exchange them.

4. Pseudocode

```
BUBBLE-SORT(A)
    for pass = 1 to A.length - 1
        for j = 1 to A.length - pass
            if A[j] > A[j + 1]
                exchange A[j] with A[j + 1]
```

5. Trace: BUBBLE-SORT([5, 3, 8, 1]), A.length = 4

Pass	Comparisons (j)	List after the pass
1	j=1: 5>3 exchange; j=2: 5>8 no; j=3: 8>1 exchange	[3, 5, 1, 8]
2	j=1: 3>5 no; j=2: 5>1 exchange	[3, 1, 5, 8]
3	j=1: 3>1 exchange	[1, 3, 5, 8]

A is now [1, 3, 5, 8].

Q18. GCD (Euclid's method)

1. Input and output

- Input: two positive whole numbers a and b.
- Output: the largest number that divides both exactly.

2. How to think about it The slow way is to test every number from the smaller one downwards. Euclid noticed a shortcut: GCD(a, b) equals GCD(b, a mod b). The remainder is smaller, so the numbers keep shrinking. When the remainder finally becomes 0, the other number is the answer. We do not know how many rounds are needed, which signals a `while` loop. (CLRS writes the same idea recursively as `EUCLID(a, b)`, which calls `EUCLID(b, a mod b)` until b is 0.)

3. Algorithm in plain English

1. Receive a and b.
2. While b is not 0: find the remainder of a divided by b, then replace a with b and b with that remainder.
3. When b reaches 0, a holds the GCD. Return it.

4. Pseudocode

```
GCD(a, b)
    while b ≠ 0
        r = a mod b
        a = b
        b = r
    return a
```

5. Trace: GCD(48, 18)

Round	a	b	r = a mod b	New a	New b
1	48	18	12	18	12

Round	a	b	r = a mod b	New a	New b
2	18	12	6	12	6
3	12	6	0	6	0

b is 0, so the loop stops. Returns 6.

Hard

Q19. Binary search

1. Input and output

- Input: an list A **already sorted in ascending order**, and a value v.
- Output: the position of v, or NIL if v is absent.

2. How to think about it Think of guessing a number between 1 and 100 when someone says "higher" or "lower". You guess the middle. If the target is bigger, the whole lower half is eliminated. If smaller, the upper half is eliminated. Each guess halves the search area, so even a million items need only about 20 checks. This only works because the list is sorted. Track the search area with two markers, low and high, and keep going while low has not passed high. If they cross, the search area is empty and v is absent.

3. Algorithm in plain English

1. Set low to 1 and high to the last position.
2. While low is not greater than high: find the middle position mid.
3. If A[mid] equals v, return mid.
4. If A[mid] is smaller than v, discard the left half by moving low to mid + 1. Otherwise discard the right half by moving high to mid - 1.
5. If the loop ends, return NIL.

4. Pseudocode

```
BINARY-SEARCH(A, v)
    low = 1
    high = A.length
    while low ≤ high
        mid = ⌊(low + high) / 2⌋
        if A[mid] == v
            return mid
        elseif A[mid] < v
            low = mid + 1
        else
            high = mid - 1
    return NIL
```

5. Trace: BINARY-SEARCH([3, 8, 12, 17, 25, 31, 40], 25)

Round	low	high	mid	A[mid]	Decision

Round	low	high	mid	A[mid]	Decision
1	1	7	4	17	17 < 25, so low = 5
2	5	7	6	31	31 > 25, so high = 5
3	5	5	5	25	equal, return 5

Returns 5. Only 3 checks were needed instead of up to 7. (For v = 10, low would end up greater than high and the procedure would return NIL.)

Q20. Balanced brackets using a stack

1. Input and output

- Input: an list s of characters, each one of `( ) [ ] { }`.
- Output: TRUE if the brackets are balanced, otherwise FALSE.

2. How to think about it The rule is: the **most recent** opening bracket must be the **first** one closed. `{ [ ( ) ] }` works because `(` opened last and closes first. `( [ ) ]` fails because `)` arrives while `[` is the most recent opener. "Last opened, first closed" is exactly how a **stack** behaves (like a pile of plates: you only touch the top). So: push every opening bracket on the stack. For each closing bracket, pop the top and check that it is the matching opener. Three things can go wrong: a closer arrives when the stack is empty, a closer does not match the top, or openers are left over at the end.

3. Algorithm in plain English

1. Start with an empty stack.
2. Read the characters one at a time.
3. If it is an opening bracket, push it onto the stack.
4. If it is a closing bracket: if the stack is empty, return FALSE. Otherwise pop the top, and if it is not the matching opener, return FALSE.
5. After the last character, return TRUE only if the stack is empty.

4. Pseudocode

```
BALANCED(s)
    let S be a new empty stack
    for i = 1 to s.length
        ch = s[i]
        if ch == "(" or ch == "[" or ch == "{"
            PUSH(S, ch)
        else
            if STACK-EMPTY(S)
                return FALSE
            top = POP(S)
            if not ((top == "(" and ch == ")") or
                    (top == "[" and ch == "]") or
                    (top == "{" and ch == "}"))
                return FALSE
    return STACK-EMPTY(S)
```

5. Trace: BALANCED("{[()]}")

ch	Action	Stack (bottom to top)
{	PUSH	{
[	PUSH	{ [
(	PUSH	{ [(
)	POP gives (, matches)	{ [
]	POP gives [, matches]	{
}	POP gives {, matches }	(empty)

The loop ends and STACK-EMPTY(S) is TRUE. Returns TRUE.

For contrast, BALANCED("([)]"): PUSH (, PUSH [, then) arrives and POP gives [. They do not match, so the procedure returns FALSE.